

Запускать так: ./run.sh

Жадный алгоритм и Pilot с bfs обходом

Напишем жадный алгоритм с лекции. Для каждого множества найдём отношение стоимости множества к ещё не покрытым элементам в нём, выберем множество с наименьшим отношением. Различие ответа такого жадного алгоритма и истинного оптимального значения будет не более чем в $H(n) = \frac{1}{n} + \frac{1}{n-1} + \dots + 1$ больше.

Действительно, пусть жадный алгоритм по очереди брал множества $S_1, S_2, \dots, S_s$, а x_i — количество новых элементов покрытых множеством S_i . Пусть $c_i = \frac{\text{cost}(S_i)}{x_i}$ — удельная цена покрытия одного элемента выбором S_i . Полная стоимость жадного решения будет равна $\sum_{i=1}^s \text{cost}(S_i) = \sum_{i=1}^s c_i x_i$. Присвоим всем элементам цену, элементам впервые добавленным множеству S_i присвоим цену c_i , тогда сумма всех стоимостей есть $\sum_{i=1}^s c_i x_i = \text{cost}(\{S_1, \dots, S_s\})$.

Рассмотрим множество A из оптимального набора. Пусть $A = \{a_1, a_2, \dots, a_k\}$, упорядочим их в порядке добавления жадным алгоритмом. Рассмотрим a_i , на момент его добавления жадным алгоритмом ещё не были покрыты $a_i, a_{i+1}, \dots, a_k$, то есть удельная цена A была $\frac{\text{cost}(A)}{k-i+1}$, при этом была выбрана наименьшая возможная, то есть $\text{cost}(a_i) \leq \frac{\text{cost}(A)}{k-i+1}$. Суммируя по всем элементам из A получается $\sum_{u \in A} \text{cost}(u) \leq \text{cost}(A) \left(1 + \frac{1}{2} + \dots + \frac{1}{|A|}\right) \leq \text{cost}(A) H(n)$. Заметим, что неравенства достаточно грубые и предполагают, что множества дизъюнкты, а оценка на стоимость элемента предполагает, что элементов других множеств нет.

Реализуем эту эвристику без ничего дополнительного. В лобовой реализации она даёт следующие метрики

102100, 30, 170, 36, 76, 192.

Что даёт 15 баллов 3, 0, 3, 3, 3, 3.

Несложное её улучшение заключается в том, что жадный алгоритм не гарантирует, что первые множества после полностью не перекрылись более поздними, так что на всякий случай попытаемся удалить каждое из множеств ответа, если вдруг все его элементы лежат в каких-то других множествах ответа. Это действительно немного улучшило метрики, они составили

102100, 30, 156, 32, 69, 181.

Что также даёт 3, 0, 3, 3, 3, 3, хоть и улучшило метрики.

Далее был написан Pilot method, мы попытались пройти все возможные способы взять или не взять множества. Так как есть гарантия на небольшое отклонение от наилучшего возможного ответа, то ветку можно закрывать, если значение жадного алгоритма делённого на H уступает наилучшему значению. Однако, как было отмечено оценка довольно грубая, так что при нахождении числа, которое может быть добавлено в подветку считать будем не само значение эвристики делённое на H , а домноженное на $\frac{1}{\alpha H^\beta}$, где α, β — настраиваемые параметры.

Уменьшение значения α позволяет не заходить в часть веток, если мы немного не добиваем до гарантии, что в ветке ответ будет не лучше, а также не верим, что оценка может оказаться точной, этот параметр скорее отвечает за то, сколько раз каждый элемент в среднем покрыт множествами наилучшего решения. Параметр β скорее отвечает за то, насколько грубая является оценка $\text{cost}(a_i) \leq \frac{\text{cost}(A)}{k-i+1}$, которая предполагает отсутствие других множеств

оптимального семейства, то есть вера в то, что H должно вычислять от значения в предположении, что $cost(a_i) \leq \frac{cost(A)}{k-i+1+C}$, то есть от самого H оставлять лишь легковесный хвост этой суммы. При смещении мы хотели бы заменить значение с $\ln(n)$ на $\ln(n+C) - \ln(C) = \ln(1 + \frac{n}{C})$, где C — количество элементов не из множества. Предположим, что n — какая-то функция от C и количество непокрытых множеств k , то есть $n = \frac{C}{k}$, а значит мы хотим заменить $\ln(\frac{C}{k}) = \ln(\frac{k+1}{k} \cdot \frac{C}{k+1}) = \ln(1 + \frac{1}{k}) + \ln(\frac{C}{k+1})$ на $\ln(1 + \frac{1}{k})$, где $\ln(\frac{C}{k+1})$ вероятно сильно больше $\ln(1 + \frac{1}{k})$, причём не в константу раз, следующая по скорости уменьшения функция это возведение в степень меньше 1, её и попробуем подобрать.

Без настройки, то есть при $\alpha, \beta = 1$ и 10 секундах на Pilot метрики составляют

96600, 25, 156, 32, 69, 181

Что даёт 18 баллов взяв все маленькие пороги 3, 3, 3, 3, 3.

Попытаемся настроить параметры. Запустимся при $\alpha = 0.2, 0.4, 0.6, 0.8$ и взглянем на ответы. При $\alpha = 0.2$ мы потеряли полезные ветки на втором тесте, но при маленьких α успели

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	96100	27	154	32	69	181
0.4	95800	25	156	32	69	181
0.6	96600	25	156	32	69	181
0.8	96600	25	156	32	69	181

больше получить на первом и третьем. α отвечало за покрытие одного элемента много раз, выведем количество таких покрытий в жадном решении: 1, 3.104, 2.150, 2.944, 3.114, 2.613, то есть маленькие значения α давали больше профита на маленьких значениях среднего перекрытия. Это несколько контринтуитивно, однако попробуем проставить α равным среднему перекрытию, делённому на некоторую константу: после экспериментов наилучшие метрики оказались на константе 5, они составили

94800, 25, 156, 32, 69, 181.

Что всё ещё 18 баллов, взяв все нижние пороги, но результат первого теста существенно улучшился.

Теперь подберём β , тоже запустимся с разными, выберем наилучшее, после чего подвтрим оптимизация α как и ранее.

β	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	102100	27	156	32	69	181
0.4	101700	27	156	32	69	181
0.6	100800	27	154	32	69	181
0.7	100000	25	153	32	69	181
0.8	98400	25	156	32	69	181
1.2	95800	25	156	32	69	181

Первый тест очень сильно зависит от α , второй и третий лучше всего показали себя при 0.7, поставим β такой и перенастроим делить в α . После настройки оказалось, что с третьим тестом нам повезло, и такое получилось при одной конкретном α , а на первый тест действительно β далёкие от 1 получаются хуже, так что вернём как было.

Запустим наш алгоритм на 60 секунд вместо 10, чтобы понять есть ли смысл отсекать какие-то ещё ветки, или чуть больше вершин дерева нам пока не помогут. Метрики на них составили:

94800, 25, 153, 32, 69, 181

В целом, от отсечения пары ветвей, которые мы при шестикратном увеличении времени скорее всего успели пройти не поможет, так что будем считать, что в этом случае мы выжали всё что можно было.

Второй способ обхода

В прошлый раз, мы решали идти или не идти по вершине в наперёд заданном порядке определённом в начале. Попробуем реализовать другой обход и настроить его.

Раньше мы вначале посортировали множества по важности и раскрывали их, теперь давайте раскрывать множества не в порядке их удельной стоимости в начале, а множество с самой большой удельной стоимостью на момент раскрытия. Это сделает процедуру раскрытия дольше, так как придётся их вычислить, но возможно выбору будут более важными.

До настройки параметров в таком обходе метрики составили

97000, 29, 156, 32, 69, 181.

Что также, как и раньше, берёт все тесты на нижние пороги.

Попробуем настроить параметры: В целом та же самая картина, аналогично тому, как и

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	95600	29	153	32	69	181
0.4	95800	29	153	32	69	181
0.6	96600	29	156	32	69	181
0.8	96600	29	156	32	69	181

раньше настроим параметры, оптимальной оказалась та же самая настройка, на ней метрики составили

94800, 29, 153, 32, 69, 181.

То есть в целом за 10 секунд был получен хороший результат на третьем тесте и провалился результат на втором, из-за того, что мы прошли меньшую часть дерева.

Также один Pilot может дать ответ, чтобы помочь второму делать более грамотные отсечения, из-за хорошего стартового решения, попытаемся запустить их оба, на 10 секунд каждый.

Порядок имеет значение, так что попробуем оба порядка. Если сначала запустить первый, а потом второй, то метрики составили

94800, 25, 153, 32, 69, 181.

Если сначала второй, а после первый, то метрики составили

94800, 25, 153, 32, 69, 181.

В целом результаты такие же, так что пожалуй, из этого метода мы выжали максимум, а улучшение не было, был просто взят лучший из ответов.

На данный момент при запуске на все 60 секунд максимального времени метрики составляют

94800, 25, 153, 32, 69, 181.

То есть вообще не изменяются от увеличения времени и дают 18 баллов 3, 3, 3, 3, 3, 3.

Локальный поиск

На многих тестах мы были близки к большому порогу, а также при наших прошлых способах обхода дерева последние решения почти всегда принимал жадник, а мы успевали назначить лишь верхние значения, попытаемся написать локальные улучшения и дожать имеющиеся ответы.